Lesson 13 Teacher's Key: A Hit That Reacts

GDevelop - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Using GDevelop's built-in "Wait X seconds" action to pause the remaining actions in an event, plus the official Flash extension for a one-line hit-flash - two genuinely new built-in tools this week, both requiring zero custom logic to use correctly.

Level 1 - Full Worked Solution

- Wait 0.1 seconds pauses only the REST of this specific event's action list - EnemiesRemaining, Score, and the sound already changed by the time the wait begins, so they're unaffected.
- The separate full-win/advance-wave events (from Lesson 11) still key off EnemiesRemaining, which changes immediately in this event - so wave/win timing is completely unaffected by today's changes.
- The Explosion object's self-cleanup event ("has existed for more than 0.3 seconds") is a fully separate, standalone event - it needs no signal or message from the collision event at all.

Highlighted lines are new this week; everything else is code you already have.

```
Bullet-Enemy collision event (revised):
Condition: Bullet is in collision with Enemy
Action: Score = Score + 10
Action: EnemiesRemaining = EnemiesRemaining - 1
Action: Play a sound (sfx_hit.wav)
Action: Create object Explosion at Enemy.X(), Enemy.Y()
Action: Flash visibility (blink) of Enemy, Duration 0.3
Action: Delete Bullet
Action: Wait 0.1 seconds
Action: Delete Enemy

Separate event (explosion self-cleanup):
Condition: Explosion has existed for more than 0.3 seconds
Action: Delete Explosion
```



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- GDevelop's Wait action is the right tool here specifically because it pauses in place within a single event's action list, without needing a Coroutine, Alarm, or manual timer variable - the most direct translation of "do this, pause, then do that" of any engine this week.
- Keeping the Explosion object's cleanup as its own condition-based event (rather than something the collision event manages) keeps every object responsible for cleaning up only itself - the same discipline bullets and enemies have followed all course.
- Level 1 deliberately uses the Flash extension's blink action rather than its color-tint action: blink just toggles visibility, so it works no matter what color the Enemy sprite is; a color tint multiplies against the sprite's existing color, and a white tint on our opaque-red Enemy would be a no-op, same underlying issue Lesson 2's Recolor was rewritten to avoid.

Level 2 / Level 3 Talking Points

- Ask students to identify exactly which action in the collision event moved (Delete Enemy, to after the Wait) versus which stayed in place (everything else) - a good comprehension check for what's actually new this week.
- For students ready for a stretch: point out the Tween behavior confirmed in the Level 3 idea, and discuss how it's a different, more flexible tool than the Flash extension - Flash is for a quick blink or color pulse, Tween is for animating any property (scale, opacity,

position) smoothly over time. If a student tries the Flash extension's color-tint action, use it as a chance to talk about why a "white" tint doesn't look white on a colored sprite (multiply blend), the same idea behind Lesson 2's sprite-swap Recolor.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
// Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
// confirm score/enemies_remaining/wave-clear/win-check all still happen
at exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
// confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
(bullet-enemy collision event - unchanged from Level 1, still uses the
Flash extension's blink action)
Condition: Bullet is in collision with Enemy
Action: Score = Score + 10
Action: EnemiesRemaining = EnemiesRemaining - 1
Action: Play a sound (sfx_hit.wav)
Action: Create object Explosion at Enemy.X(), Enemy.Y()
Action: Flash visibility (blink) of Enemy, Duration 0.3
Action: Delete Bullet
Action: Wait 0.1 seconds
Action: Delete Enemy

(Explosion's Create event - Level 3 expanding-burst Tween, NEW; add the
Tween behavior to Explosion first: Behaviors tab > Add a behavior > Tween)
Action: Start object scale Tween for Explosion, from 0.5 to 1.5, over 0.3
seconds, easing Linear

(separate event, self-cleanup - unchanged from Level 1)
Condition: Explosion has existed for more than 0.3 seconds
Action: Delete Explosion
```

Common Student Mistakes

- Placing Delete Enemy BEFORE the Wait action instead of after, which defeats the whole point (the enemy would still vanish instantly).
- Installing the Flash extension but forgetting to add the Flash behavior to Enemy specifically (Behaviors tab > Add a behavior > Flash) - the extension only makes the behavior available project-wide; the blink action still won't appear for Enemy until the behavior is actually attached to it.
- Building the Explosion self-cleanup condition incorrectly (e.g. checking the wrong object, or a typo'd name), leaving explosions on screen permanently.